Project Title: QA Metrics Analysis and Optimization Proposals - Streamline Mobile App 2.0

1. Trend Analysis and Inferred Problems

Based on the data collected over the last three months, the following trends and their potential underlying causes are observed:

Negative Trends and Areas of Concern:

- **Declining Test Reliability (from 90% to 80%):** This suggests that tests are failing inconsistently (generating false positives) or that there are stability issues in the testing environments.
- **High Defect Leakage:** A large number of problems (18 in month 3) continue to escape into the production environment. This correlates with Test Coverage, which, although improving, consistently remains below the acceptable target level (80%+).
- **Improvement in Mean Time to Detect (MTTD) and Mean Time to Repair (MTTR):** The team is finding and fixing errors faster. However, the current speed of resolution may not be enough to prevent users from abandoning the solution when facing defects in production.
- **Drop in User Adoption Rate (from 40% to 38%):** This is a critical sign that users are frustrated with the application's problems and prefer to abandon the product.

Positive Trends and their Nuances:

- **Misleading Increase in Customer Satisfaction (CSAT):** Although satisfaction seems to rise, it occurs simultaneously with a drop in adoption. This indicates bias: frustrated users simply abandon the application and do not participate in surveys, leaving only the opinions of the more tolerant users.

2. Proposed Solutions and their Expected Impact

To mitigate the identified problems, the following strategic actions are proposed:

Proposal A: Aggressively increase automated test coverage

- **Actions:** Dedicate a *sprint* exclusively to increasing test coverage, aiming for 80% or more. Furthermore, the "Definition of Done" (DoD) must be updated to ensure that new code always includes automated tests.
- **Impacted Metrics:** Test Coverage, Defect Leakage, and Test Cycle Time.
- **Explanation:** Low coverage is a fundamental root cause for errors reaching the customer. By ensuring that most of the code is automatically tested, production defects will be reduced and future test cycles will be accelerated.

Proposal B: Improve the quality of Code Reviews

- **Actions:** Implement mandatory checklists during code reviews to validate the existence and quality of unit tests before approving any integration.
- **Impacted Metrics:** Test Coverage and Defect Leakage.
- **Explanation:** It is likely that developers are releasing code without coverage because the current process allows it. By introducing strict standards in the review, technical responsibility is encouraged and defect leakage is stopped at the source.

Proposal C: Stabilize Flaky Tests and optimize CI/CD

- **Actions:** Analyze the patterns of the tests that produce false positives most frequently and suggest a *sprint* focused on resolving the automation technical debt.
- **Impacted Metrics:** Test Reliability and Test Cycle Time.
- **Explanation:** The drop in reliability indicates poor quality in test engineering. Solving this ensures consistent results, eliminates bottlenecks in validations, and ensures more reliable deployments.

Proposal D: Introduce proactive Beta testing with real users

- **Actions:** Analyze the characteristics of the users who have abandoned the app and invite similar profiles to participate in surveys, interviews, or beta tests before official releases.
- **Impacted Metrics:** User Adoption Rate and Customer Satisfaction (CSAT).
- **Explanation:** The best way to validate why the application is not delivering the expected value is to ask the users. This will allow for the discovery of usability, performance, and missing feature issues before they reach the production environment, thus stopping the drop in adoption.